CAMPUSRIDE

Campus Transport Management System

PROJECT REPORT

Prepared by: Gollapalli Eshwar

Campus transport information management and GPS location-sharing prototype

Table of Contents

Abstract

1. Introduction

2. Problem Statement

3. Objectives

4. Scope of the Project

5. System Modules

6. System Architecture and Technologies

7. GPS Tracking Workflow

8. Data Management and CRUD

9. Expected Benefits

10. Limitations and Security Considerations

11. Future Enhancements

12. Conclusion

Appendix: Short Viva Answers

Abstract

CampusRide is a browser-based prototype designed to make campus transport management easier. It focuses on organizing bus information and related transport details, such as drivers, routes, stops and timings. The prototype also demonstrates a GPS-sharing workflow in which a driver selects a registered bus, grants location permission, and shares the latest available coordinates so the owner can check them. The current version is hosted on Hatchable and uses its backend with a PostgreSQL database. It is a prototype and should not be described as a completed MERN-stack implementation.

1. Introduction

Campus transportation involves maintaining information about buses, drivers, routes, stops and schedules. When information is handled manually or across separate records, it can take time to find details and keep them updated. CampusRide proposes a single, simple interface for managing transport information and checking the latest location shared by a bus device.

The project is intended as a prototype for demonstrating the main workflow and identifying what would be required for a more complete transport management application.

2. Problem Statement

Manual transport records can be difficult to maintain, search and update. Transport staff may need to check different records to find bus or driver details. In addition, the owner may not have an easy way to check the latest bus location. CampusRide aims to reduce this effort by organizing transport information and providing a location-sharing workflow.

3. Objectives

- Reduce repetitive manual work in campus transport management.
- Keep bus and driver information organized.
- Maintain route, stop and timing details in one application.
- Demonstrate sharing of the latest bus coordinates through device GPS permission.
- Make it easier for an owner to review the latest location submitted by a driver’s device.
- Provide a foundation that can be expanded with authentication, maps and additional transport features.

4. Scope of the Project

The prototype scope includes a browser-based interface, transport record management concepts, and a GPS Tracking section where a registered bus can be selected and location sharing started. The latest submitted coordinates can be retrieved from the backend and shown to the owner as location information with a map link.

The prototype does not yet represent a fully production-ready transport platform. Full user authentication, role-based access control, a complete interactive live map, and a full MERN-stack migration are future enhancements.

5. System Modules

5.1 Bus Management: provides a place to maintain bus records and identify the bus whose location is being shared.

5.2 Driver Management: represents driver-related information associated with campus transport operations.

5.3 Route and Stop Management: organizes the planned paths and stops used by campus buses.

5.4 Schedule Management: stores or displays timing information for transport planning.

5.5 GPS Tracking: allows the device user to select a bus, grant location permission and start sharing location updates.

5.6 Owner Location View: retrieves the latest stored coordinates for buses that have shared a location. The owner can refresh the list and open a map link to inspect coordinates.

6. System Architecture and Technologies

The current prototype is hosted on Hatchable. The user interacts with the browser-based interface; application logic sends requests to backend API endpoints; and the backend stores or retrieves records in PostgreSQL. The GPS-sharing endpoint accepts location updates and provides the latest saved bus locations.

Current implementation: browser-based frontend using HTML, CSS and JavaScript, Hatchable backend/API functionality, and PostgreSQL data storage.

Planned MERN architecture (if required by the project specification): React.js for the user interface, Node.js for the server runtime, Express.js for API routing, and MongoDB for document storage. This is a proposed architecture and should only be described as implemented after the application has actually been migrated.

7. GPS Tracking Workflow

1. The driver opens the GPS Tracking section.

2. The driver selects a registered bus from the list.

3. The browser asks for location permission; the driver must allow it.

4. The device obtains location coordinates and sends updates to the backend.

5. The backend stores the latest submitted coordinates against the selected bus.

6. The owner opens the location view and refreshes or checks the latest shared coordinates.

If the driver denies location permission, the browser cannot provide the device’s current location, so new GPS updates cannot be sent. Location sharing also depends on device settings, browser support and network availability.

8. Data Management and CRUD

CRUD means Create, Read, Update and Delete. These are common operations for managing records.

- Create: add a new transport record, such as a bus.
- Read: view stored bus, driver, route or timing information.
- Update: change details when a bus or other record changes.
- Delete: remove a record that is no longer required.

A database helps organize records and makes it easier for an application to retrieve and update information than relying only on scattered manual records.

9. Expected Benefits

- Reduces repetitive manual handling of transport information.
- Keeps related records organized in one application.
- Makes it easier to view and update stored details.
- Allows the owner to inspect the latest location submitted by a bus device.
- Provides a base for adding maps, notifications, reports and stronger access controls.

10. Limitations and Security Considerations

The current version is a prototype, not a production-ready transport service. GPS coordinates represent the latest location shared by the device and should not be presented as guaranteed continuous tracking. A full interactive live map and route navigation are not yet complete.

Before real-world deployment, the application should include secure login, separate driver and owner permissions, validation of bus identifiers, protection of location endpoints, HTTPS, error handling, and a clear policy for how long location data is retained. Bus locations are sensitive operational information and should not be made publicly accessible.

11. Future Enhancements

- Implement the full MERN stack if required by the project brief.
- Add secure owner and driver login with role-based permissions.
- Display bus locations on an interactive map with route and stop markers.
- Add arrival, delay and route-change notifications.
- Add data validation, automated tests, logging and better error messages.
- Improve mobile responsiveness and test the system with multiple buses and devices.

12. Conclusion

CampusRide demonstrates a practical approach to simplifying campus transport management by organizing transport records and sharing the latest available bus coordinates. The prototype provides a starting point for a larger system. With secure authentication, a complete map interface, thorough testing and any required migration to the MERN stack, it could be developed further into a more complete campus transport application.

Appendix: Short Viva Answers

Main objective: CampusRide aims to reduce manual work, organize transport details and help the owner check the latest bus location shared through GPS.

API: An API allows different parts of an application, such as the frontend and backend, to communicate and exchange data.

GPS tracking vs route management: GPS shows where a bus is now; route management defines the planned path and stops.

Database: A database stores and organizes information so it can be accessed and updated.

Frontend vs backend: The frontend is what users see and interact with; the backend processes requests and manages data.